

BÁO CÁO MÔ HÌNH ĐỀ XUẤT

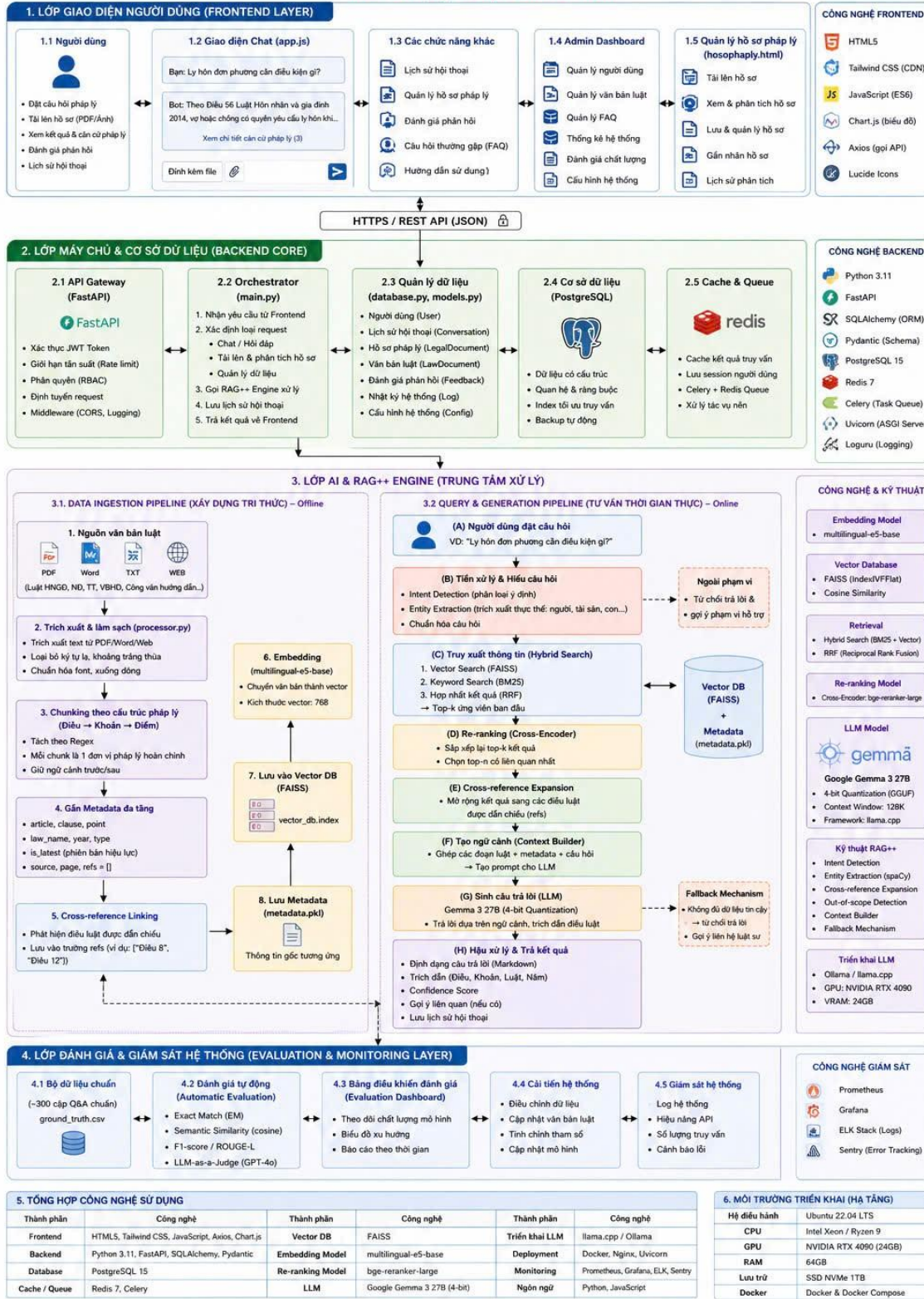
ỨNG DỤNG CÔNG NGHỆ RAG++ XÂY DỰNG CHATBOT TRỢ GIÚP PHÁP LÝ THÔNG MINH VỀ LUẬT HÔN NHÂN VÀ GIA ĐÌNH

Sinh viên thực hiện	Nguyễn Văn Hồ (MSSV: 222520)
Lớp / Khóa	DH22KPM01 – Khóa 10
GVHD	ThS. Trần Văn Thiện
Trường / Khoa	ĐH Nam Cần Thơ – Khoa CNTT

MÔ HÌNH ĐỀ XUẤT CHI TIẾT

SƠ ĐỒ KIẾN TRÚC CHI TIẾT MÔ HÌNH ĐỀ XUẤT – CHATBOT TRỢ GIÚP PHÁP LÝ RAG++ (Hệ thống tư vấn Luật Hôn nhân và Gia đình)

LLM sử dụng: Google Gemma 3 27B



Lớp 1 – Giao diện Người dùng (Frontend)



Lớp frontend đóng vai trò là cầu nối giữa người dùng và hệ thống AI pháp lý. Thành phần này được xây dựng dưới dạng ứng dụng web với các công nghệ hiện đại như HTML5, Tailwind CSS, JavaScript ES6, Axios API và Chart.js.

1.1 Người dùng

Người dùng có thể tương tác với hệ thống thông qua các chức năng:

- Đặt câu hỏi pháp lý;
- Tải lên hồ sơ pháp lý dưới dạng PDF hoặc ảnh;
- Theo dõi lịch sử hội thoại;
- Xem kết quả tư vấn;
- Đánh giá chất lượng phản hồi.

Điều này giúp hệ thống không chỉ hoạt động như chatbot thông thường mà còn đóng vai trò như trợ lý hỗ trợ pháp lý trực tuyến.

1.2 Giao diện hội thoại

Khu vực chat được thiết kế theo mô hình hội thoại thời gian thực (real-time conversational interface), cho phép:

- Nhập câu hỏi;
- Nhận phản hồi từ AI;
- Đính kèm tài liệu pháp lý;
- Hiện thị nguồn tham chiếu điều luật.

Ví dụ, khi người dùng đặt câu hỏi liên quan đến ly hôn đơn phương, chatbot sẽ phân tích nội dung và trả về căn cứ pháp luật tương ứng trong Luật Hôn nhân và Gia đình năm 2014.

1.3 Các chức năng hỗ trợ

Ngoài chatbot chính, hệ thống còn cung cấp:

- Lịch sử trò chuyện;
- Quản lý hồ sơ pháp lý;
- Đánh giá phản hồi;
- FAQ và hướng dẫn sử dụng.

Những chức năng này góp phần nâng cao trải nghiệm người dùng và khả năng khai thác hệ thống.

1.4 Dashboard quản trị

Admin Dashboard hỗ trợ quản trị viên:

- Quản lý người dùng;
- Quản lý văn bản luật;
- Quản lý FAQ;
- Theo dõi thống kê hệ thống;
- Đánh giá hiệu năng chatbot.

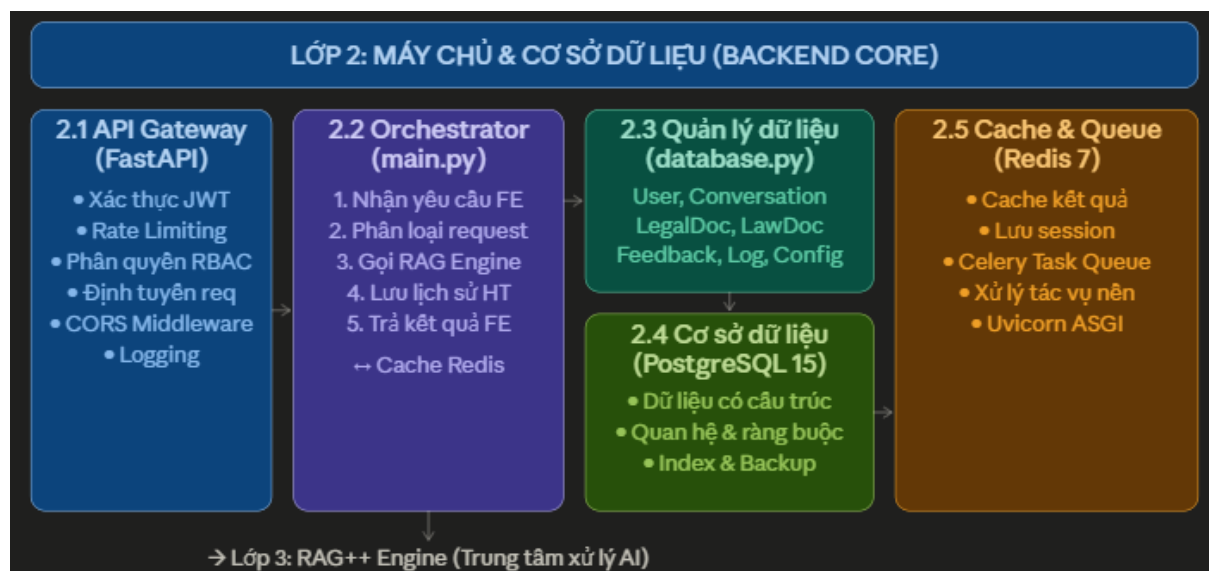
Đây là thành phần quan trọng trong việc vận hành và bảo trì hệ thống thực tế.

1.5 Quản lý hồ sơ pháp lý

Hệ thống cho phép lưu trữ và phân loại hồ sơ pháp lý nhằm hỗ trợ:

- Tra cứu hồ sơ;
- Gắn nhãn hồ sơ;
- Theo dõi lịch sử xử lý;
- Phân tích dữ liệu hồ sơ.

Lớp 2 – Máy chủ & Cơ sở dữ liệu (Backend Core)



Lớp backend đóng vai trò xử lý nghiệp vụ, quản lý dữ liệu và điều phối luồng xử lý giữa frontend và hệ thống AI.

2.1 API Gateway

API Gateway được xây dựng bằng FastAPI, đảm nhiệm:

- Xử lý các HTTP Request;
- Xác thực JWT Token;
- Giới hạn tần suất truy cập (Rate Limiting);
- Định tuyến API;
- Middleware CORS và Logging.

Kiến trúc này giúp hệ thống bảo mật hơn và có khả năng mở rộng tốt.

2.2 Orchestrator

Orchestrator là trung tâm điều phối nghiệp vụ của backend, thực hiện:

- Tiếp nhận yêu cầu từ frontend;
- Phân loại yêu cầu;
- Xử lý logic hội thoại;
- Gọi các pipeline AI;
- Lưu trữ lịch sử hội thoại;
- Trả kết quả về frontend.

Có thể xem đây là “bộ não điều phối” toàn bộ luồng xử lý của hệ thống.

2.3 Quản lý dữ liệu

Thành phần này quản lý:

- Người dùng;
- Hội thoại;
- Hồ sơ pháp lý;
- Văn bản luật;
- Nhật ký hệ thống;
- Cấu hình ứng dụng.

Dữ liệu được ánh xạ thông qua ORM SQLAlchemy và xác thực bằng Pydantic Schema.

2.4 Cơ sở dữ liệu PostgreSQL

PostgreSQL được sử dụng để lưu trữ:

- Dữ liệu cấu trúc;
- Thông tin người dùng;
- Lịch sử hội thoại;
- Metadata văn bản luật;
- Chỉ mục truy vấn.

Hệ thống cũng hỗ trợ backup tự động nhằm đảm bảo an toàn dữ liệu.

2.5 Cache và Queue

Redis và Celery được tích hợp nhằm:

- Cache kết quả truy vấn;
- Tăng tốc phản hồi;
- Quản lý hàng đợi tác vụ nền;
- Giảm tải cho hệ thống chính.

Lớp 3 – AI & RAG++ Engine (Trung tâm xử lý)



3.1 Data Ingestion Pipeline (Offline)

Pipeline này chịu trách nhiệm xây dựng kho tri thức pháp luật.

Bước 1: Thu thập nguồn dữ liệu

Nguồn dữ liệu bao gồm:

- PDF luật;
- Word;
- TXT;
- Website pháp luật;
- Công văn hướng dẫn.

Bước 2: Tiền xử lý dữ liệu

Hệ thống thực hiện:

- Trích xuất văn bản;
- Loại bỏ ký tự nhiễu;
- Chuẩn hóa Unicode;
- Làm sạch dữ liệu.

Bước 3: Chunking theo cấu trúc pháp lý

Khác với RAG truyền thống, hệ thống chia văn bản theo:

- Điều luật;
- Khoản;
- Điểm;
- Cấu trúc ngữ nghĩa pháp lý.

Cách tiếp cận này giúp tăng độ chính xác truy xuất.

Bước 4: Gắn Metadata

Mỗi đoạn dữ liệu được bổ sung metadata như:

- Tên văn bản luật;
- Điều luật;
- Năm ban hành;
- Hiệu lực;
- Nguồn dữ liệu.

Bước 5: Cross-reference Linking

Hệ thống phát hiện và liên kết các điều luật liên quan nhằm hình thành mạng tri thức pháp lý.

Bước 6: Embedding

Các đoạn văn bản được chuyển thành vector ngữ nghĩa bằng mô hình embedding multilingual-e5-base.

Bước 7–8: Lưu Vector DB và Metadata

Vector embedding được lưu vào FAISS, trong khi metadata được lưu riêng để hỗ trợ truy xuất nhanh và chính xác.



3.2 Query & Generation Pipeline (Online)

Đây là pipeline xử lý truy vấn thời gian thực.

(A) Người dùng đặt câu hỏi

Ví dụ:

“Ly hôn đơn phương cần điều kiện gì?”

(B) Tiền xử lý và hiểu câu hỏi

Hệ thống thực hiện:

- Intent Detection;
- Entity Extraction;
- Chuẩn hóa câu hỏi.

Ví dụ, hệ thống có thể nhận diện:

- Chủ thể: vợ/chồng;
- Hành động: ly hôn;
- Ngữ cảnh: đơn phương.

(C) Hybrid Search

Hệ thống sử dụng kết hợp:

- Semantic Search bằng FAISS;
- Keyword Search bằng BM25;
- Reciprocal Rank Fusion (RRF).

Mục tiêu là nâng cao độ chính xác truy xuất tài liệu pháp lý.

(D) Re-ranking bằng Cross-Encoder

Các kết quả truy xuất được sắp xếp lại bằng mô hình Cross-Encoder nhằm chọn ra các đoạn luật có mức liên quan cao nhất.

(E) Cross-reference Expansion

Hệ thống mở rộng truy xuất sang các điều luật được viện dẫn hoặc liên quan trực tiếp.

Đây là điểm nâng cấp đặc trưng của kiến trúc RAG++.

(F) Context Builder

Các đoạn luật phù hợp cùng metadata được tổng hợp thành ngữ cảnh đầu vào cho LLM.

(G) Sinh câu trả lời bằng LLM

Mô hình Gemma 3 27B được sử dụng để:

- Sinh phản hồi ngôn ngữ tự nhiên;
- Giải thích điều luật;
- Tóm tắt căn cứ pháp lý;
- Đề xuất hướng tư vấn phù hợp.

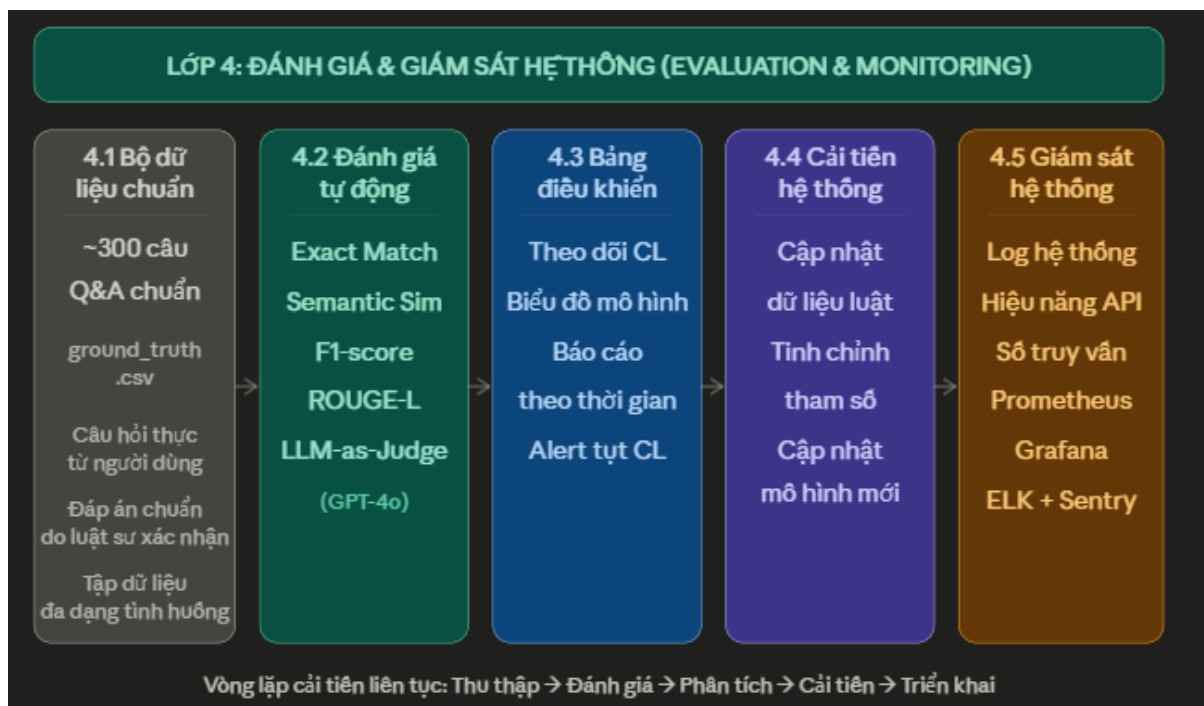
(H) Hậu xử lý và trả kết quả

Kết quả cuối cùng bao gồm:

- Nội dung tư vấn;
- Trích dẫn điều luật;
- Độ tin cậy;
- Gợi ý liên quan;
- Lưu lịch sử hội thoại.

Nếu độ tin cậy thấp, hệ thống sẽ kích hoạt cơ chế fallback để hạn chế trả lời sai.

Lớp 4: Đánh giá & Giám sát hệ thống



Lớp này hỗ trợ kiểm thử, đánh giá và tối ưu chất lượng chatbot.

4.1 Bộ dữ liệu chuẩn

Sử dụng tập câu hỏi chuẩn phục vụ đánh giá độ chính xác.

4.2 Đánh giá tự động

Các metric đánh giá gồm:

- Exact Match;
- Semantic Similarity;
- F1-score;
- ROUGE-L;
- LLM-as-a-Judge.

4.3 Dashboard đánh giá

Hiển thị:

- Chất lượng mô hình;
- Biểu đồ xu hướng;
- Báo cáo theo thời gian.

4.4 Cải tiến hệ thống

Cho phép:

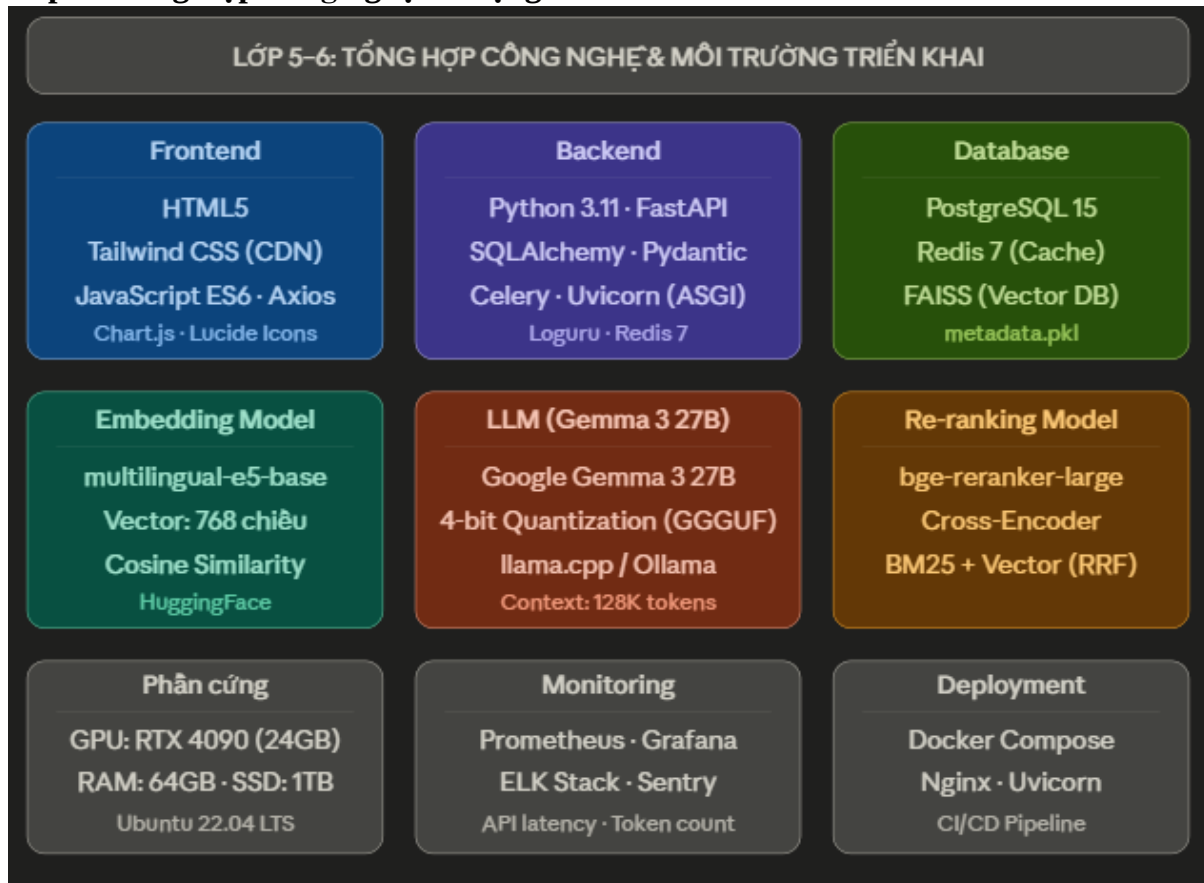
- Điều chỉnh dữ liệu;
- Tinh chỉnh tham số;
- Cập nhật mô hình AI.

4.5 Giám sát hệ thống

Sử dụng Prometheus, Grafana, ELK Stack và Sentry để:

- Giám sát hiệu năng;
- Theo dõi API;
- Phát hiện lỗi;
- Ghi log hệ thống.

Lớp 5: Tổng hợp công nghệ sử dụng



Hình trên mô tả **toàn bộ stack công nghệ** và **môi trường triển khai thực tế** của hệ thống Chatbot trợ giúp pháp lý ứng dụng mô hình RAG++. Đây là lớp cuối cùng trong kiến trúc hệ thống, đóng vai trò hiện thực hóa toàn bộ mô hình AI thành một ứng dụng có thể vận hành ổn định trong môi trường thực tế.

Kiến trúc được chia thành nhiều nhóm công nghệ chính gồm:

- Frontend
- Backend
- Database
- Embedding Model
- LLM
- Re-ranking Model
- Hạ tầng phần cứng
- Monitoring
- Deployment

1. Frontend – Giao diện người dùng

Frontend là lớp giao tiếp trực tiếp giữa người dùng và hệ thống chatbot.

Công nghệ sử dụng

HTML5

HTML5 được sử dụng để xây dựng cấu trúc giao diện web.

Tailwind CSS (CDN)

Tailwind CSS là framework CSS utility-first giúp:

- Thiết kế giao diện nhanh;
- Responsive trên nhiều thiết bị;

- Dễ tùy biến;
- Giảm thời gian viết CSS thủ công.

Hệ thống sử dụng bản CDN nhằm triển khai nhanh trong môi trường nghiên cứu và phát triển.

JavaScript ES6

JavaScript ES6 xử lý:

- Tương tác động;
- Gửi yêu cầu đến backend;
- Hiển thị phản hồi chatbot;
- Xử lý upload file;
- Cập nhật dữ liệu thời gian thực.

Axios

Axios là thư viện HTTP Client dùng để:

- Gửi API request từ frontend đến FastAPI backend;
- Nhận dữ liệu JSON;
- Quản lý lỗi kết nối.

Chart.js

Chart.js hỗ trợ:

- Hiển thị biểu đồ thống kê;
- Dashboard đánh giá hệ thống;
- Phân tích hiệu năng chatbot.

Lucide Icons

Thư viện icon giúp giao diện:

- Trực quan hơn;
- Dễ sử dụng;
- Tăng trải nghiệm người dùng.

2. Backend – Máy chủ xử lý trung tâm

Backend là nơi xử lý toàn bộ logic nghiệp vụ và điều phối AI pipeline.

Công nghệ sử dụng

Python 3.11

Python là ngôn ngữ chính của hệ thống do:

- Hỗ trợ mạnh cho AI/ML;
- Nhiều thư viện NLP;
- Dễ phát triển backend.

Phiên bản 3.11 cho hiệu năng tốt hơn các phiên bản trước.

FastAPI

FastAPI là framework backend hiện đại dùng để:

- Xây dựng REST API;
- Xử lý request bất đồng bộ;
- Tăng tốc độ phản hồi;
- Tự động sinh Swagger API Docs.

Đây là framework rất phù hợp cho hệ thống AI thời gian thực.

SQLAlchemy

SQLAlchemy là ORM giúp:

- Kết nối PostgreSQL;
- Thao tác database bằng Python object;

- Giảm viết SQL thủ công;
- Dễ bảo trì hệ thống.

Pydantic

Pydantic hỗ trợ:

- Kiểm tra dữ liệu đầu vào;
- Validate request/response;
- Chuẩn hóa schema API.

Điều này giúp hệ thống ổn định và giảm lỗi dữ liệu.

Celery

Celery là hệ thống xử lý tác vụ nền (background task queue), dùng cho:

- Xử lý embedding;
- Import dữ liệu luật;
- Gửi email;
- Tác vụ AI mất nhiều thời gian.

Uvicorn (ASGI)

Uvicorn là ASGI Server dùng để chạy FastAPI.

Vai trò:

- Xử lý asynchronous request;
- Tăng throughput;
- Hỗ trợ websocket/chat realtime.

Loguru

Loguru hỗ trợ:

- Ghi log hệ thống;
- Theo dõi lỗi;
- Debug pipeline AI.

Redis 7

Redis trong backend được dùng làm:

- Cache dữ liệu;
- Message broker cho Celery;
- Lưu session tạm thời;
- Tăng tốc hệ thống.

3. Database – Hệ quản trị dữ liệu

Đây là lớp lưu trữ dữ liệu của hệ thống.

PostgreSQL 15

PostgreSQL là cơ sở dữ liệu quan hệ chính dùng để lưu:

- Người dùng;
- Lịch sử hội thoại;
- Hồ sơ pháp lý;
- Metadata luật;
- Nhật ký hệ thống.

Ưu điểm:

- Ổn định;
- ACID compliance;
- Hiệu năng cao;
- Phù hợp hệ thống lớn.

Redis 7 (Cache)

Redis ngoài vai trò backend còn hoạt động như lớp cache:

- Giảm truy vấn database;
- Tăng tốc phản hồi chatbot.

FAISS (Vector DB)

FAISS là thư viện vector database do Facebook AI phát triển.

Vai trò:

- Lưu embedding vector;
- Tìm kiếm semantic similarity;
- Hỗ trợ retrieval trong RAG++.

Đây là thành phần cực kỳ quan trọng trong pipeline AI.

metadata.pkl

File metadata.pkl lưu:

- Thông tin điều luật;
- Điều/khoản;
- Văn bản nguồn;
- Mapping với vector embedding.

Giúp chatbot giải thích và truy xuất chính xác.

4. Embedding Model – Mô hình vector hóa ngữ nghĩa

multilingual-e5-base

Đây là mô hình embedding dùng để chuyển văn bản thành vector ngữ nghĩa.

Vai trò:

- Biểu diễn ý nghĩa câu hỏi;
- Tìm kiếm semantic search;
- So sánh mức độ liên quan giữa câu hỏi và điều luật.

Vector 768 chiều

Mỗi đoạn văn bản được biểu diễn bằng vector 768 dimensions.

Vector càng biểu diễn tốt thì khả năng retrieval càng chính xác.

Cosine Similarity

Dùng để tính:

- Độ tương đồng giữa câu hỏi và tài liệu luật;
- Xác định đoạn luật phù hợp nhất.

HuggingFace

Là nền tảng cung cấp:

- Mô hình embedding;
- NLP model;
- Open-source AI ecosystem.

5. LLM – Large Language Model

Đây là “bộ não sinh ngôn ngữ” của chatbot.

Google Gemma 3 27B

Gemma 3 27B là mô hình ngôn ngữ lớn của Google với khoảng 27 tỷ tham số.

Vai trò:

- Sinh câu trả lời;
- Giải thích luật;
- Tóm tắt pháp lý;
- Tư vấn hội thoại tự nhiên.

4-bit Quantization (GGUF)

Kỹ thuật quantization giúp:

- Giảm VRAM sử dụng;
- Chạy được trên RTX 4090;
- Tăng tốc inference.

GGUF là định dạng tối ưu cho llama.cpp/Ollama.

llama.cpp / Ollama

Hai công nghệ dùng để:

- Chạy LLM local;
- Quản lý model;
- Tối ưu inference.

Điều này giúp hệ thống:

- Không phụ thuộc cloud API;
- Tăng tính riêng tư dữ liệu pháp lý.

Context 128K Tokens

Cho phép LLM đọc:

- Văn bản luật dài;
- Nhiều điều luật cùng lúc;
- Hồ sơ pháp lý lớn.

Đây là lợi thế quan trọng trong bài toán pháp lý.

6. Re-ranking Model – Mô hình xếp hạng lại kết quả

Sau khi retrieval lấy ra các đoạn luật, hệ thống cần sắp xếp lại mức độ liên quan.

bge-reranker-large

Đây là Cross-Encoder model dùng để:

- Đánh giá mức liên quan thực sự;
- Chọn ra context tốt nhất cho LLM.

Khác với embedding search, reranker hiểu sâu hơn ngữ cảnh.

Cross-Encoder

Cross-Encoder đưa:

- Câu hỏi;
- Đoạn luật

vào cùng một mô hình để tính relevance score chính xác hơn.

BM25 + Vector (RRF)

Hệ thống sử dụng Hybrid Search:

- BM25 → keyword search;
- Vector Search → semantic search;
- RRF → hợp nhất kết quả.

Điều này giúp tăng accuracy retrieval đáng kể.

7. Phần cứng (Hardware)

GPU RTX 4090 (24GB)

GPU dùng để:

- Chạy Gemma 27B;
- Sinh embedding;
- Tăng tốc inference AI.

24GB VRAM phù hợp với mô hình quantized 4-bit.

RAM 64GB

RAM lớn hỗ trợ:

- Xử lý nhiều request;
- Lưu cache;
- Chạy database và AI cùng lúc.

SSD 1TB

SSD NVMe giúp:

- Tăng tốc đọc ghi vector DB;
- Tăng tốc load model AI.

Ubuntu 22.04 LTS

Hệ điều hành server ổn định cho AI deployment.

8. Monitoring – Giám sát hệ thống

Prometheus

Thu thập metrics hệ thống:

- CPU;
- RAM;
- API latency;
- GPU usage.

Grafana

Hiển thị dashboard monitoring trực quan.

ELK Stack

Bao gồm:

- Elasticsearch;
- Logstash;
- Kibana.

Dùng để:

- Phân tích log;
- Truy vết lỗi;
- Theo dõi hoạt động chatbot.

Sentry

Theo dõi exception và lỗi runtime theo thời gian thực.

9. Deployment – Triển khai hệ thống

Docker Compose

Quản lý nhiều service:

- Backend;
- Database;
- Redis;
- Monitoring;
- AI model.

Nginx

Nginx hoạt động như:

- Reverse proxy;
- Load balancer;
- HTTPS gateway.

Uvicorn

Server chạy FastAPI production.

CI/CD Pipeline

Tự động:

- Build;
- Test;
- Deploy hệ thống.

Giúp cập nhật nhanh và giảm lỗi vận hành.